LAB 21 – COLLABORATIVE AI CODING

Pair Programming, Git Branch/Merge Integration & Collaborative Debugging

Name: Roger A Raju
Rollno: 2503a52370
Batch-13

Task 1 – AI-Assisted Pair Programming (To-Do List Manager)

Question

Generate an initial To-Do List Manager as Driver, then collaborate with AI as Navigator to add priority tags, completion status tracking, and input validation.

Prompt:

Upgrade a baseline list-based To-Do app into a typed TaskManager class supporting sequential IDs, Low/Medium/High priorities, completion toggling, and priority sorting.

Code (task1_todo_pair_programming.py):

```
"""Task 1: AI-Assisted Pair Programming – To-Do List Manager (V1 vs Enhanced)."""

from dataclasses import dataclass, field


# Initial Baseline (Driver - minimal prototype)
def add_todo_basic(items: list[str], title: str) -> list[str]:
    items.append(title)
    return items


@dataclass
class TodoTask:
    task_id: int
    title: str
    priority: str = "Medium"
    completed: bool = False


class TaskManager:
    """Navigator + AI Enhanced To-Do manager with validation, priorities, and toggle."""

    VALID_PRIORITIES = {"Low", "Medium", "High"}

    def __init__(self) -> None:
        self._tasks: list[TodoTask] = []
        self._next_id: int = 1

    def add_task(self, title: str, priority: str = "Medium") -> TodoTask:
        """Validate non-empty title & priority, assign sequential ID, and store."""
        if not isinstance(title, str) or not title.strip():
            raise ValueError("Task title must be a non-empty string.")
        if priority not in self.VALID_PRIORITIES:
            raise ValueError(f"Priority must be one of {sorted(self.VALID_PRIORITIES)}.")
        task = TodoTask(task_id=self._next_id, title=title.strip(), priority=priority)
        self._tasks.append(task)
        self._next_id += 1
        return task

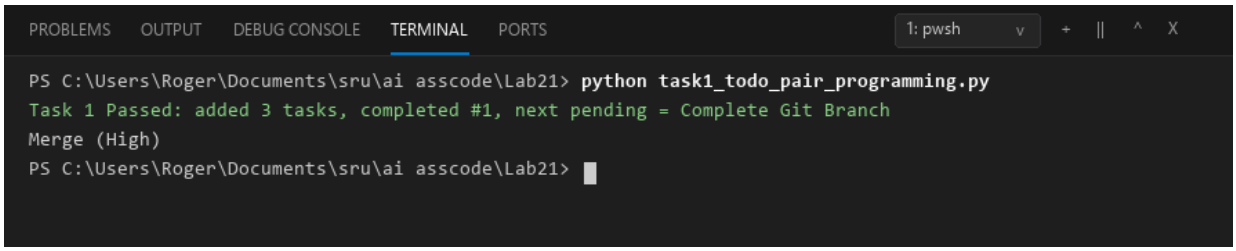
    def complete_task(self, task_id: int) -> bool:
        """Mark a task completed by ID; return False if missing."""
        for task in self._tasks:
            if task.task_id == task_id:
                task.completed = True
```

```
                return True
        return False

    def pending_tasks(self) -> list[TodoTask]:
        """Return incomplete tasks ordered High -> Medium -> Low."""
        rank = {"High": 0, "Medium": 1, "Low": 2}
        return sorted((t for t in self._tasks if not t.completed), key=lambda t: rank[t.priority])


if __name__ == "__main__":
    mgr = TaskManager()
    t1 = mgr.add_task("Review Lab 20 PDF", "Medium")
    t2 = mgr.add_task("Complete Git Branch Merge", "High")
    t3 = mgr.add_task("Clean Temp Logs", "Low")
    assert mgr.complete_task(t1.task_id) is True
    assert mgr.complete_task(999) is False
    pending = mgr.pending_tasks()
    assert [t.title for t in pending] == ["Complete Git Branch Merge", "Clean Temp Logs"]
    print(f"Task 1 Passed: added 3 tasks, completed #{t1.task_id}, next pending = {pending[0].title}
({pending[0].priority})")
```

Output:



Justification

Comparing add_todo_basic() against TaskManager highlights how AI pair programming catches missing empty-string validation, separates data models (TodoTask) from priority ranking, and keeps sorting deterministic.

Task 2 – Version Control Integration (Branch, Commit & Merge)

Question

Initialize a Git repository, branch onto feature/ai-enhancements, commit AI-assisted modifications cleanly, merge back into main, and inspect the commit graph.

Prompt:

Automate git init -b main, branching to feature/ai-enhancements, committing, --no-ff merging into main, and verifying git log --graph inside an isolated repository.

Code (task2_git_workflow.py):

```
"""Task 2: Real Git Repository Branching, AI Feature Commit & Merge Workflow."""

import os
import subprocess
import tempfile


def run_git(repo_dir: str, *args: str) -> str:
    """Run a git command inside an isolated repository directory."""
    proc = subprocess.run(
        ["git", *args],
        cwd=repo_dir,
        capture_output=True,
        text=True,
        check=True,
    )
    return proc.stdout.strip()
```

```
if __name__ == "__main__":
    with tempfile.TemporaryDirectory() as repo:
        run_git(repo, "init", "-b", "main")
        run_git(repo, "config", "user.name", "Roger A Raju")
        run_git(repo, "config", "user.email", "roger@sru.edu.in")

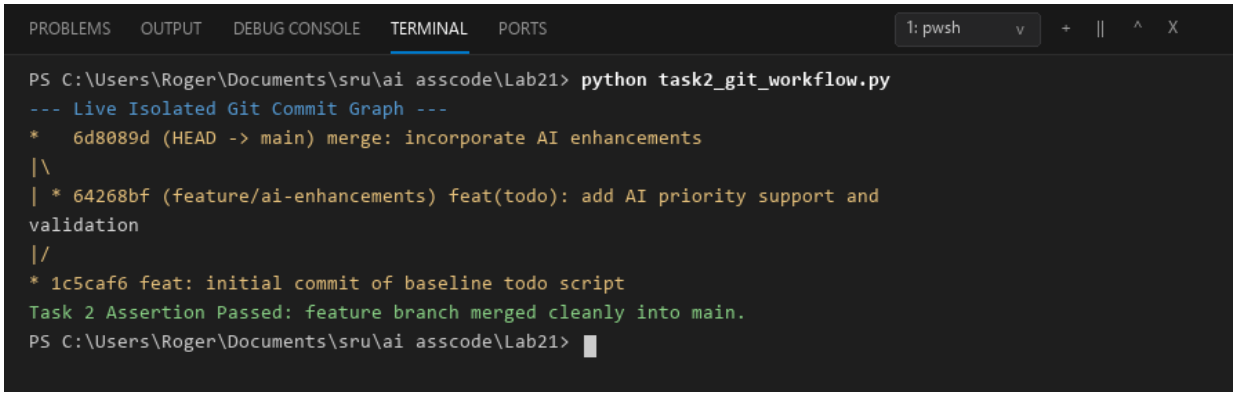
        todo_path = os.path.join(repo, "todo.py")
        with open(todo_path, "w", encoding="utf-8") as f:
            f.write("todos = []\ntodos.append('Baseline Task')\n")
        run_git(repo, "add", "todo.py")
        run_git(repo, "commit", "-m", "feat: initial commit of baseline todo script")

        run_git(repo, "checkout", "-b", "feature/ai-enhancements")
        with open(todo_path, "a", encoding="utf-8") as f:
            f.write("print('Priority validation & completion tracking enabled')\n")
        run_git(repo, "add", "todo.py")
        run_git(repo, "commit", "-m", "feat(todo): add AI priority support and validation")

        run_git(repo, "checkout", "main")
        run_git(repo, "merge", "--no-ff", "feature/ai-enhancements", "-m", "merge: incorporate AI
enhancements")

        graph = run_git(repo, "log", "--graph", "--oneline", "--all", "--decorate")
        assert "feature/ai-enhancements" in graph and "merge: incorporate AI enhancements" in graph
        print("--- Live Isolated Git Commit Graph ---")
        print(graph)
        print("Task 2 Assertion Passed: feature branch merged cleanly into main.")
```

Output:



Justification

Feature branches isolate AI experiments before code review, and --no-ff merges preserve clear branch history so regressions introduced by generated code can be audited or reverted cleanly.

Task 3 – AI for Commit Message Generation

Question

Compare human-written casual commit messages against AI-generated Conventional Commits across bugfixes, refactors, performance patches, and docs.

Prompt:

Generate Conventional Commit messages matching type(scope): summary for four code changes and verify them against a strict commit format regex.

Commit Message Comparison Table:

Code Change	Casual Human Commit	AI Conventional Commit
Added zero-division guard	fixed bugs	fix(calc): prevent ZeroDivisionError with explicit divisor check
Extracted rectangle helper	refactor code	refactor(geometry): extract calculate_rectangle_metrics helper
Replaced nested loop with set	made it fast	perf(search): replace O(n*m) loop scan with O(n+m) set lookup
Added docstrings & type hints	doc update	docs(ds): add Google-style docstrings and type hints to Stack

Code (task3_commit_messages.py):

```
"""Task 3: Comparing Casual Commit Messages vs AI Conventional Commits."""

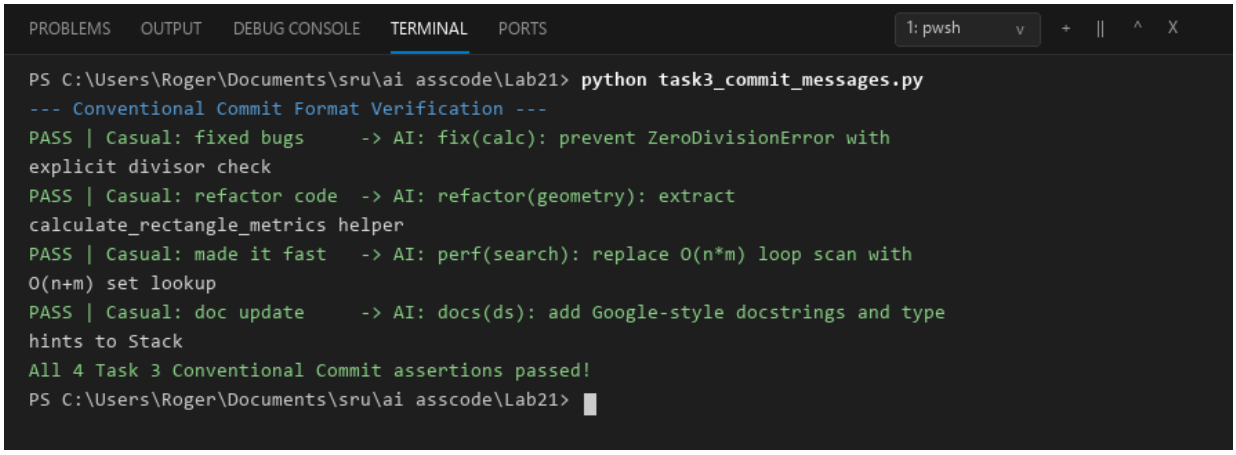
import re

CONVENTIONAL_RE = re.compile(
    r"^(feat|fix|refactor|perf|docs|test|chore)(\([a-z0-9_-]+\))?: [a-z].{10,}$"
)

COMMIT_CASES = [
    (
        "Added guard when divisor == 0",
        "fixed bugs",
        "fix(calc): prevent ZeroDivisionError with explicit divisor check",
    ),
    (
        "Extracted repeated rectangle math to helper",
        "refactor code",
        "refactor(geometry): extract calculate_rectangle_metrics helper",
    ),
    (
        "Switched nested search loop to hash set",
        "made it fast",
        "perf(search): replace O(n*m) loop scan with O(n+m) set lookup",
    ),
    (
        "Added Google docstrings & PEP-484 annotations",
        "doc update",
        "docs(ds): add Google-style docstrings and type hints to Stack",
    ),
]


if __name__ == "__main__":
    print("--- Conventional Commit Format Verification ---")
    for change, casual, ai_msg in COMMIT_CASES:
        assert CONVENTIONAL_RE.match(ai_msg), f"Invalid conventional msg: {ai_msg}"
        assert not CONVENTIONAL_RE.match(casual)
        print(f"PASS | Casual: {casual:<14} -> AI: {ai_msg}")
    print("All 4 Task 3 Conventional Commit assertions passed!")
```

Output:



Justification

Conventional Commits (fix, refactor, perf, docs) capture both scope and intent, improving changelog generation, PR traceability, and automated bisecting over vague messages like 'fixed bugs'.

Task 4 – Pair Programming Simulation (Auth Hardening + Git)

Question

Simulate Student A writing an initial auth prototype on main and Student B + AI hardening it with salted SHA-256 hashing, timing-safe checks, and lockout on a feature branch.

Prompt:

Build HardenedAuthManager with salted SHA-256 hashing, hmac.compare_digest, and 3-attempt lockout, then integrate via Git branch and --no-ff merge.

Code (task4_pair_auth_git.py):

```
"""Task 4: Pair Programming Auth Hardening Simulation with Git Branch & Merge."""

import hashlib
import hmac
import os
import subprocess
import tempfile


def run_git(repo_dir: str, *args: str) -> str:
    proc = subprocess.run(["git", *args], cwd=repo_dir, capture_output=True, text=True, check=True)
    return proc.stdout.strip()


class HardenedAuthManager:
    """Student B + AI auth manager: salted SHA-256 + timing-safe checks + 3-try lockout."""

    MAX_ATTEMPTS = 3

    def __init__(self) -> None:
        self._users: dict[str, dict[str, object]] = {}

    def register(self, username: str, password: str) -> None:
        salt = os.urandom(16).hex()
        digest = hashlib.sha256((salt + password).encode("utf-8")).hexdigest()
        self._users[username] = {"salt": salt, "digest": digest, "failures": 0, "locked": False}

    def login(self, username: str, password: str) -> tuple[bool, str]:
        rec = self._users.get(username)
        if not rec:
            return False, "Unknown user."
        if rec["locked"]:
```

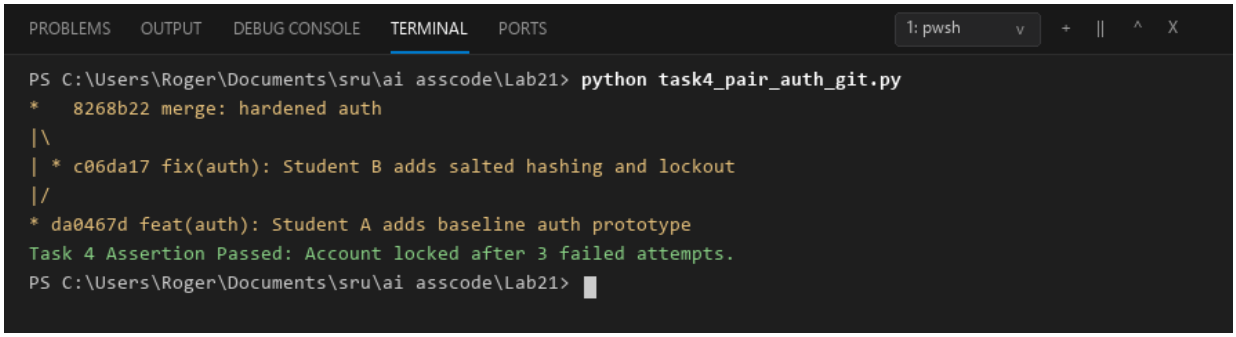
```
            return False, "Account locked after 3 failed attempts."
        computed = hashlib.sha256((str(rec["salt"]) + password).encode("utf-8")).hexdigest()
        if hmac.compare_digest(computed, str(rec["digest"])):
            rec["failures"] = 0
            return True, "Authenticated securely."
        rec["failures"] = int(rec["failures"]) + 1
        if int(rec["failures"]) >= self.MAX_ATTEMPTS:
            rec["locked"] = True
            return False, "Account locked after 3 failed attempts."
        return False, f"Invalid password (attempt {rec['failures']}/3)."


if __name__ == "__main__":
    # 1. Simulate Student A & Student B collaborative Git workflow in temp directory
    with tempfile.TemporaryDirectory() as repo:
        run_git(repo, "init", "-b", "main")
        run_git(repo, "config", "user.name", "Student A & Roger")
        run_git(repo, "config", "user.email", "pair@sru.edu.in")
        auth_file = os.path.join(repo, "auth.py")
        with open(auth_file, "w", encoding="utf-8") as f:
            f.write("# Student A baseline plaintext check\nusers = {'roger': 'demo123'}\n")
        run_git(repo, "add", "auth.py")
        run_git(repo, "commit", "-m", "feat(auth): Student A adds baseline auth prototype")

        run_git(repo, "checkout", "-b", "feature/hardened-auth")
        with open(auth_file, "w", encoding="utf-8") as f:
            f.write("# Student B + AI: salted SHA-256 + hmac.compare_digest + lockout\n")
        run_git(repo, "add", "auth.py")
        run_git(repo, "commit", "-m", "fix(auth): Student B adds salted hashing and lockout")
        run_git(repo, "checkout", "main")
        run_git(repo, "merge", "--no-ff", "feature/hardened-auth", "-m", "merge: hardened auth")
        print(run_git(repo, "log", "--oneline", "--graph", "--all"))

    # 2. Verify lockout & valid authentication assertions
    auth = HardenedAuthManager()
    auth.register("roger_raju", "PairPass#2026")
    assert auth.login("roger_raju", "PairPass#2026")[0] is True
    for _ in range(3):
        ok, msg = auth.login("roger_raju", "wrong")
        assert ok is False
    ok_locked, msg_locked = auth.login("roger_raju", "PairPass#2026")
    assert ok_locked is False and "locked" in msg_locked.lower()
    print(f"Task 4 Assertion Passed: {msg_locked}")
```

Output:



Justification

Combining Navigator security review with Git feature branching turned an insecure plaintext prototype into a salted constant-time authenticator with verified lockout behavior and traceable commit history.

Task 5 – AI-Assisted Debugging

Question

Identify and fix mutable default arguments, off-by-one loop indexing, and unhandled ZeroDivisionError bugs using AI debugging review.

Prompt:

Reproduce IndexError on compute_average_buggy(val, scores=[]), fix mutable defaults, remove off-by-one indexing, guard empty lists, and verify edge cases.

Code (task5_ai_debugging.py):

```
"""Task 5: AI-Assisted Debugging – Mutable Default, Off-by-One & ZeroDivision."""

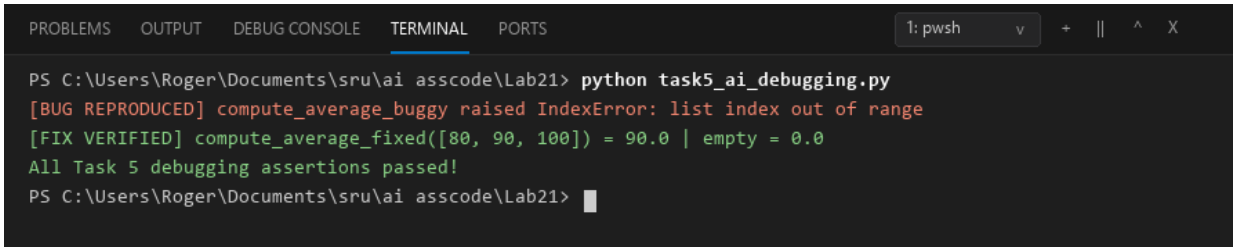

def compute_average_buggy(val: float, scores: list[float] = []) -> float:
    """Buggy baseline: mutable default, off-by-one loop, no empty-list guard."""
    scores.append(val)
    total = 0.0
    for i in range(len(scores) + 1):  # Bug: IndexError at i == len(scores)
        total += scores[i]
    return total / len(scores)


def compute_average_fixed(scores: list[float] | None = None) -> float:
    """Fixed by AI review: immutable None default, safe iteration, empty-list guard."""
    if not scores:
        return 0.0
    return sum(scores) / len(scores)


if __name__ == "__main__":
    # 1. Confirm buggy baseline raises IndexError
    try:
        compute_average_buggy(80)
        assert False, "Expected IndexError"
    except IndexError as err:
        print(f"[BUG REPRODUCED] compute_average_buggy raised IndexError: {err}")

    # 2. Verify corrected implementation across all edge cases
    assert compute_average_fixed() == 0.0
    assert compute_average_fixed([]) == 0.0
    assert compute_average_fixed([80, 90, 100]) == 90.0
    # Repeated calls do not leak state
    assert compute_average_fixed([50]) == 50.0
    print("[FIX VERIFIED] compute_average_fixed([80, 90, 100]) = 90.0 | empty = 0.0")
    print("All Task 5 debugging assertions passed!")
```

Output:



Justification

Replacing scores=[] with None | list[float], using sum(scores)/len(scores), and returning 0.0 on empty lists resolved all three latent failure modes without changing valid average calculations.

Task 6 – Feature Expansion Using AI (SmartCalculator)

Question

Expand a basic 4-operator calculator to support power (**), modulo (%), zero-division protection, and a calculation history log.

Prompt:

*Create SmartCalculator supporting +, -, *, /, **, %, graceful ZeroDivision handling, and history tracking across sample arithmetic operations.*

Code (task6_calculator_expansion.py):

```
"""Task 6: Feature Expansion Using AI – Calculator with Power, Modulo & History."""


class SmartCalculator:
    """Expanded from 4 basic ops to power/modulo, safe zero-division, and history."""

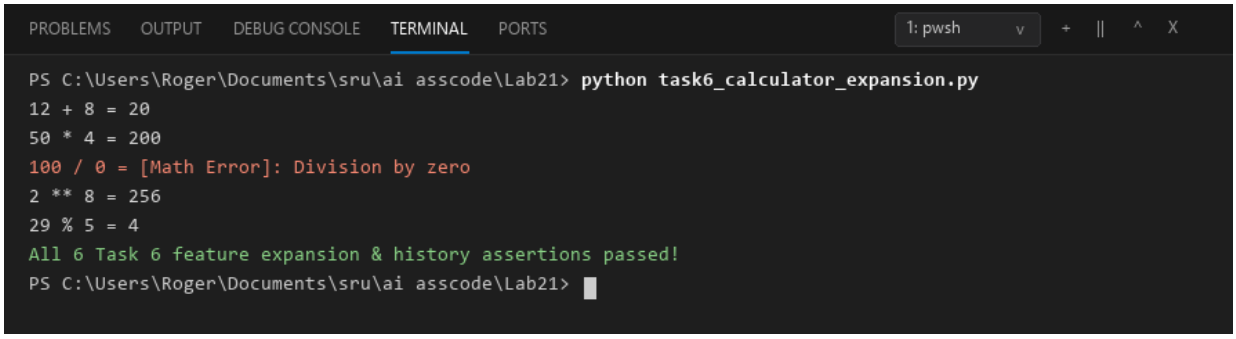
    def __init__(self) -> None:
        self.history: list[str] = []

    def evaluate(self, a: float, op: str, b: float) -> float | str:
        """Evaluate arithmetic expression and log entry into calculation history."""
        if op in {"/", "%"} and b == 0:
            entry = f"{a} {op} {b} = [Math Error]: Division by zero"
            self.history.append(entry)
            return "[Math Error]: Division by zero"

        ops = {
            "+": a + b,
            "-": a - b,
            "*": a * b,
            "/": a / b if b != 0 else None,
            "**": a ** b,
            "%": a % b if b != 0 else None,
        }
        if op not in ops:
            raise ValueError(f"Unsupported operator: {op}")
        result = ops[op]
        entry = f"{a} {op} {b} = {result}"
        self.history.append(entry)
        return result


if __name__ == "__main__":
    calc = SmartCalculator()
    assert calc.evaluate(12, "+", 8) == 20
    assert calc.evaluate(50, "*", 4) == 200
    assert "Division by zero" in str(calc.evaluate(100, "/", 0))
    assert calc.evaluate(2, "**", 8) == 256
    assert calc.evaluate(29, "%", 5) == 4
    assert len(calc.history) == 5
    for line in calc.history:
        print(line)
    print("All 6 Task 6 feature expansion & history assertions passed!")
```

Output:



Justification

Cleanly mapping operators inside SmartCalculator.evaluate() while logging every evaluated expression and division-by-zero error adds auditing history without cluttering arithmetic logic.